

DEVOPS MASTER COURSE

Volume 6D

AWS Containers, Serverless & Event-Driven Architecture

Chapters 31-40

**ECR | ECS | FARGATE | EKS | LAMBDA | API GATEWAY | SQS/SNS | EVENTBRIDGE | STEP
FUNCTIONS**

Production container platforms, serverless applications, event-driven integration, labs,
troubleshooting, and interviews

2026 Edition

About This Part

Volume 6D moves from infrastructure and data platforms into modern application execution. It explains how teams package software as immutable container images, operate services on Amazon ECS and Amazon EKS, use AWS Fargate to remove host management, build event-driven functions with AWS Lambda, expose APIs safely, decouple systems with messaging, route domain events, and orchestrate distributed workflows.

The central engineering theme is controlled coupling. Production systems should isolate failures, scale individual components independently, provide durable recovery paths, expose clear telemetry, and preserve security boundaries from source code to runtime. Every chapter therefore connects service configuration to delivery pipelines, identity, networking, observability, cost, and incident response.

Scope and Safety

- Use a sandbox AWS account or an approved training environment. Configure budgets and expiry tags before creating clusters, NAT gateways, load balancers, container tasks, or managed Kubernetes resources.
- Use IAM roles, temporary credentials, private networking, encrypted transport, managed secrets, and least-privilege resource policies. Never place access keys or secrets in images, task definitions, environment files, source repositories, or Lambda deployment packages.
- Validate current regional availability, service quotas, supported platform versions, runtime versions, Kubernetes versions, pricing, and feature constraints in official AWS documentation before production use.
- Build idempotent consumers and explicit retry policies. Event-driven services commonly deliver at least once, so duplicate handling is a business requirement rather than an optional optimization.
- Delete chargeable lab resources after collecting evidence. EKS control planes, load balancers, NAT gateways, provisioned concurrency, and continuously running tasks can generate charges even when applications are idle.

How to Use the Labs

- Define the service-level objective, failure boundary, deployment strategy, security model, and rollback path before provisioning.
- Tag resources with owner, environment, application, cost center, data classification, and expiry.
- Capture commands, resource identifiers, logs, metrics, traces, screenshots, and test results as operational evidence.
- Inject controlled failures: unhealthy tasks, poison messages, permission denial, timeout, dependency failure, and partial workflow errors.
- Repeat the final architecture with Infrastructure as Code after proving the design manually.

Table of Contents

Chapter 31 Amazon ECR and Container Supply-Chain Engineering

Chapter 32 Amazon ECS Production Engineering

Chapter 33 AWS Fargate Serverless Containers

Chapter 34 Amazon EKS and Kubernetes Operations

Chapter 35 AWS Lambda Production Engineering

Chapter 36 Amazon API Gateway Engineering

Chapter 37 Amazon SQS and Amazon SNS Messaging

Chapter 38 Amazon EventBridge Integration Engineering

Chapter 39 AWS Step Functions Workflow Orchestration

Chapter 40 Enterprise Event-Driven Application Architecture and Capstone

Navigation Heading styles are applied throughout so Word and LibreOffice can generate a field-based table of contents and provide document-outline navigation.

VOLUME 6D | CHAPTER 31

Chapter 31 - Amazon ECR and Container Supply-Chain Engineering

From source commit to trusted, immutable runtime artifact

Learning Objectives

- Explain OCI container images, layers, manifests, tags, digests, and registries.
- Design private ECR repositories, repository policies, encryption, replication, lifecycle controls, and vulnerability scanning.
- Implement immutable promotion and provenance-oriented release practices.
- Authenticate Docker or compatible clients securely and troubleshoot push/pull failures.
- Build a container supply-chain gate suitable for ECS, EKS, and Fargate.

31.1 Why the Registry Is a Security Boundary

A container registry is not merely a file store. It is the control point between software construction and software execution. Source code becomes an image; the image becomes a deployment artifact; and the digest identifies the exact bytes that a runtime pulls. If the registry accepts untrusted images, permits mutable release tags, or lacks lifecycle and scanning controls, every downstream runtime inherits that risk.

Production teams therefore treat the registry as part of the software supply chain. CI systems push images, security controls evaluate them, release automation promotes approved digests, and runtime roles receive pull-only permissions. Human users rarely need direct push access to production repositories.

31.2 Image Anatomy and Identity

Concept	Meaning	Production consequence
Layer	Content-addressed filesystem change shared across images.	Small stable layers improve cache reuse; secrets copied into any layer remain recoverable even if later deleted.
Tag	Human-readable mutable pointer such as <code>api:1.8.2</code> or <code>api:stable</code> .	Useful for discovery but unsafe as the only deployment identity.
Digest	Cryptographic content identifier such as <code>sha256:...</code>	Pins the exact image and supports deterministic rollback and promotion.
Manifest	Metadata that references image layers and platform information.	Multi-architecture manifests can select suitable images for different CPU architectures.
Repository	Logical collection of related images.	Repository boundaries simplify IAM, lifecycle, replication, and ownership.

Production rule Deploy immutable digests. Tags may communicate release intent, but the deployment record must capture the resolved digest so a rollback retrieves identical bytes.

31.3 ECR Control Plane

Amazon Elastic Container Registry provides private repositories integrated with IAM. A repository can use resource policies for cross-account access, KMS-backed encryption choices, image scanning, lifecycle policies, and replication. Authentication uses a short-lived registry authorization flow rather than permanent Docker passwords.

Control	Why it matters	Recommended implementation
IAM identity policy	Controls which principals can call ECR APIs.	Separate CI push permissions, runtime pull permissions, and administrator lifecycle permissions.
Repository policy	Supports repository-level and cross-account authorization.	Allow a designated deployment account or organization path; avoid public principals.
Image scanning	Finds known software vulnerabilities.	Scan on push or continuously according to risk; block promotion on defined severity and exception policy.
Lifecycle policy	Removes or archives stale images based on rules.	Preview rules before application; retain active release digests and rollback windows.
Replication	Copies images across Regions or accounts.	Use for DR, regional autonomy, and centralized build/decentralized runtime patterns.
CloudTrail and EventBridge	Records and reacts to repository activity.	Alert on policy changes, unusual deletes, scan findings, and unexpected pushes.

31.4 Secure Build and Promotion Pattern

```
Developer commit
-> CI build in isolated runner
-> unit/integration tests
-> generate SBOM and image metadata
-> push candidate image to build repository
-> vulnerability and policy gates
-> sign/attest approved digest
-> copy or retag approved digest into release repository
-> deploy by digest to ECS/EKS/Lambda container runtime
```

Promotion should move the same digest between environments rather than rebuild from the same source. Rebuilding can introduce a new base image, dependency, timestamp, or compiler output. The promoted artifact must therefore be traceable to the source revision, build job, tests, approvals, and deployment change record.

31.5 CLI Workflow

```

REGION=ap-south-1
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
REPO=orders-api

aws ecr create-repository \
  --repository-name "$REPO" \
  --image-scanning-configuration scanOnPush=true \
  --image-tag-mutability IMMUTABLE \
  --region "$REGION"

aws ecr get-login-password --region "$REGION" | \
  docker login --username AWS --password-stdin \
  "$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com"

docker build -t "$REPO:1.0.0" .
docker tag "$REPO:1.0.0" \
  "$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/$REPO:1.0.0"
docker push "$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/$REPO:1.0.0"

aws ecr describe-images --repository-name "$REPO" \
  --query "imageDetails[].[digest:imageDigest,tags:imageTags,pushed:imagePushedAt]"

```

31.6 Lifecycle Policy Design

Lifecycle rules are evaluated as a policy, not as independent cleanup scripts. Protect release tags and active digests, then remove untagged build artifacts and old non-production images. Always use the preview capability before application. A cleanup rule that does not reflect rollback requirements can convert a routine rollback into a rebuild and security-review emergency.

```

{
  "rules": [
    {
      "rulePriority": 10,
      "description": "Expire untagged images after 14 days",
      "selection": {
        "tagStatus": "untagged",
        "countType": "sinceImagePushed",
        "countUnit": "days",
        "countNumber": 14
      },
      "action": {"type": "expire"}
    }
  ]
}

```

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
docker push returns no basic auth credentials	Login token absent, expired, or generated for another Region/account.	Check current identity, repository Region, registry hostname, and Docker credential helper state.	Repeat get-login-password for the exact registry and verify IAM actions.

Symptom	Likely cause	Investigation	Corrective action
Task cannot pull image	Execution role lacks ECR pull permissions, network path is unavailable, or image tag/digest is wrong.	Inspect ECS stopped reason or kubelet events; test VPC endpoints/NAT and repository existence.	Grant minimum pull permissions, restore egress/endpoints, and pin the correct digest.
Scan gate blocks every release	Base image is old, severity threshold is unrealistic, or findings lack exception workflow.	Review scan findings, package path, fix availability, and exploitability.	Update base/dependencies and document time-bound exceptions with compensating controls.
Lifecycle policy deleted rollback image	Rules ignored release retention or active deployment references.	Review CloudTrail, policy preview history, deployment digest, and replica repositories.	Restore from replica/archive if available; redesign retention around deployment inventory.

Hands-on Lab 31: Build a Trusted ECR Repository

Objective Create an immutable ECR repository, push a versioned image, capture its digest, scan it, preview cleanup, and prove a deployment can reference the digest.

Prerequisites

- Docker or compatible container CLI
- AWS CLI v2 and an IAM role with scoped ECR permissions
- A minimal local web application and Dockerfile

Implementation

1. Create an ECR repository with immutable tags and scan-on-push enabled.
2. Authenticate the client to the repository registry.
3. Build a small image with a version label and non-root runtime user.
4. Push version 1.0.0 and record the image digest.
5. Attempt to overwrite the immutable tag and record the expected failure.
6. Review image scan findings and document the release decision.
7. Create and preview a lifecycle policy for untagged images.
8. Pull the image by digest and run it locally.

Validation Evidence

- Repository configuration export
- Recorded digest and successful digest pull
- Scan output and risk decision
- Lifecycle preview showing only intended candidates

Cleanup

- Stop the local container.
- Delete nonessential images and the repository after confirming no later lab depends on it.

Stretch challenge Configure cross-account pull access to a dedicated runtime account and verify that the runtime role can pull but cannot push or delete images.

Interview Questions and Model Answers

1. Why should production deployments use an image digest instead of only a tag?

A digest is content-addressed and identifies the exact artifact. A tag can be moved or overwritten, so tag-only deployment weakens reproducibility, auditability, and rollback integrity.

2. How would you separate CI and runtime permissions?

CI receives push, describe, and scan-related permissions for build repositories. Runtime execution roles receive only the actions needed to obtain authorization and pull layers. Delete, policy, lifecycle, and replication permissions remain administrative.

3. What belongs in a container supply-chain gate?

Source revision linkage, reproducible build metadata, tests, SBOM, vulnerability evaluation, secret scanning, policy checks, approval evidence, signature or attestation, and digest-based promotion.

4. How do lifecycle policies create operational risk?

They can remove rollback artifacts or base layers if retention rules do not align with deployed versions, release cadence, incident investigation, and regulatory retention.

Chapter Summary

- ECR is a supply-chain security boundary, not only an image store.
- Tags are convenient labels; digests are deployment identities.
- Separate build, promotion, runtime, and administration permissions.
- Scanning, lifecycle preview, replication, audit events, and artifact provenance are production requirements.

Official AWS References

Amazon ECR User Guide: <https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html>

ECR image scanning: <https://docs.aws.amazon.com/AmazonECR/latest/userguide/image-scanning.html>

ECR lifecycle policies: <https://docs.aws.amazon.com/AmazonECR/latest/userguide/LifecyclePolicies.html>

VOLUME 6D | CHAPTER 32

Chapter 32 - Amazon ECS Production Engineering

Operating container services without managing a Kubernetes control plane

Learning Objectives

- Model ECS clusters, task definitions, tasks, services, capacity providers, and deployments.
- Design secure networking, IAM roles, service discovery, load balancing, autoscaling, and secrets delivery.
- Choose EC2, Fargate, or mixed capacity according to workload constraints.
- Implement health checks, deployment rollback, observability, and operational access.
- Troubleshoot placement, image pull, application health, and scaling failures.

32.1 ECS Resource Model

Amazon Elastic Container Service is a regional container orchestrator. You describe a workload in a task definition, run one or more tasks, and use a service when a desired task count must be maintained. A cluster is the logical scheduling boundary. Capacity providers connect the scheduler to compute capacity such as Fargate or EC2 Auto Scaling groups.

Resource	Responsibility	Operational decision
Task definition	Versioned specification for containers, CPU/memory, ports, volumes, roles, logs, secrets, and health checks.	Treat revisions as release artifacts; never edit running tasks in place.
Task	A running instantiation of a task definition.	Use standalone tasks for finite work; services for continuously available workloads.
Service	Maintains desired tasks and coordinates deployments.	Configure minimum healthy percent, maximum percent, rollback, load balancer, and autoscaling.
Cluster	Logical grouping and capacity association.	Separate environments or trust zones when policy, ownership, or blast radius requires it.
Capacity provider	Strategy for where tasks run and how infrastructure scales.	Mix base capacity, On-Demand, Spot, EC2, or Fargate according to availability and cost.

32.2 IAM Roles: Execution Versus Application

The task execution role is used by the ECS agent or Fargate platform for actions such as pulling an ECR image, retrieving referenced secrets, and writing logs. The task role is assumed by application code inside the container. Keeping these roles separate prevents an application from inheriting infrastructure bootstrap privileges and enables per-service least privilege.

Common failure Granting permissions to the EC2 instance profile does not automatically make them appropriate for each task. Prefer task roles so every service receives only its own application permissions.

32.3 Networking and Service Connectivity

The awsvpc network mode gives each task an elastic network interface and security-group identity. This makes tasks first-class VPC endpoints and is the standard model for Fargate. Public applications typically place tasks in private subnets behind an Application Load Balancer. Outbound access uses NAT or VPC endpoints; inbound rules allow only the load-balancer security group. Internal service communication can use private DNS and ECS Service Connect or service discovery.

```
Internet
-> Route 53
-> public ALB in two or more AZs
-> target group
-> ECS tasks in private subnets
-> private database/cache endpoints
-> CloudWatch Logs, metrics, traces

Security groups:
ALB: inbound 443 from approved clients; outbound to task port
Tasks: inbound task port from ALB SG only; outbound to required dependencies
```

32.4 Deployment Controls

Mechanism	Use	Risk control
Rolling deployment	Replaces tasks within service deployment limits.	Use health checks, adequate surge capacity, and deployment circuit breaker rollback.
Blue/green deployment	Shifts traffic between old and new task sets.	Use test listener, bake time, alarms, and explicit rollback criteria.
Versioned task definition	Captures image digest and runtime configuration.	Keep previous revisions available and record the deployed revision in change evidence.
Container health check	Evaluates process-level health inside task.	Avoid shallow checks that pass while dependencies or critical paths fail.
ALB target health	Evaluates network application path.	Coordinate grace periods with startup time to avoid replacement loops.

32.5 Capacity and Scaling

Service Auto Scaling changes desired task count using target tracking, step scaling, or scheduled policies. EC2-backed clusters also require infrastructure capacity scaling. Capacity providers can connect ECS to Auto Scaling groups and maintain a target capacity. Scaling signals should represent service pressure: request rate per target, queue backlog per task, latency, saturation, or domain throughput. CPU alone is often insufficient.

32.6 Observability and Operations

- Send structured application logs to CloudWatch Logs or an approved telemetry pipeline. Include request, trace, deployment, tenant, and error identifiers.
- Emit task-level and business metrics. Container Insights can add infrastructure visibility but does not replace application SLO metrics.
- Use distributed tracing across load balancer, service, queue, function, and database boundaries.
- Use ECS Exec for approved, audited break-glass diagnostics instead of inbound SSH. Keep production containers minimal and immutable.
- Alarm on failed deployments, stopped tasks, unhealthy targets, throttling, saturation, and error-budget burn.

```
aws ecs describe-services \
  --cluster prod-apps \
  --services orders-api \
  --query "services[0].
{desired:desiredCount,running:runningCount,pending:pendingCount,events:events[0:5]}"

aws ecs list-tasks --cluster prod-apps --service-name orders-api
aws ecs describe-tasks --cluster prod-apps --tasks <task-arn>
```

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Task remains PENDING	No compatible capacity, subnet IP exhaustion, resource mismatch, or placement constraint.	Review service events, task CPU/memory, capacity provider, available instances, subnet IPs.	Add compatible capacity/IP space or correct task requirements and strategy.
Task stops immediately	Application exits, essential sidecar fails, image cannot start, or secret/config is invalid.	Inspect stoppedReason, container reason/exit code, and log stream.	Correct command, configuration, secret permissions, architecture, or dependency startup.
ALB target stays unhealthy	Wrong port/path, SG denial, startup too slow, or health endpoint fails.	Curl from VPC path, inspect target reason, task logs, listener/target configuration.	Fix port mapping/path/security groups and configure realistic grace period.
Deployment never stabilizes	New tasks fail while old tasks are constrained by deployment percentages or capacity.	Review deployment state, events, health checks, and capacity headroom.	Rollback, add surge capacity, or adjust deployment limits and health criteria.

Hands-on Lab 32: Deploy a Resilient ECS Service

Objective Deploy a private ECS web service behind an ALB, use distinct execution and task roles, enable logs, and prove automatic replacement and rollback.

Prerequisites

- A VPC with two public and two private subnets

- An ECR image digest from Lab 31
- Permissions to create ECS, IAM roles, ALB resources, logs, and autoscaling

Implementation

9. Create a cluster and associate an approved capacity provider.
10. Create CloudWatch log group with retention and encryption settings.
11. Register a task definition using the ECR digest, awsvpc mode, non-root user, health check, execution role, and restricted task role.
12. Create an ALB, listener, target group, and security groups using SG-to-SG rules.
13. Create an ECS service across at least two AZs with deployment rollback enabled.
14. Confirm healthy traffic and structured logs.
15. Stop one task manually and verify service replacement.
16. Deploy a deliberately unhealthy revision and verify rollback behavior.
17. Add target-tracking autoscaling and run a controlled load test.

Validation Evidence

- Service deployment and task-definition revision
- Healthy targets across AZs
- Replacement event after task termination
- Failed deployment evidence and successful rollback
- Scaling metrics and post-test desired count

Cleanup

- Scale service to zero if supported by later labs or delete service.
- Delete ALB/listener/target group, log group, cluster, roles, and security groups when no longer needed.

Stretch challenge Add ECS Service Connect for a second internal service and prove that only the intended namespace and port are reachable.

Interview Questions and Model Answers

1. What is the difference between an ECS task role and task execution role?

The execution role supports platform bootstrap actions such as image pull, logs, and referenced secrets. The task role is the application identity used by SDK calls inside the container.

2. Why can an ECS service scale tasks successfully while requests still fail?

Scaling may use a poor signal, new tasks may not become healthy, downstream dependencies may saturate, load-balancer routing may be wrong, or connection pools may not scale with task count.

3. When would you choose ECS over EKS?

Choose ECS when AWS-native orchestration, simpler operations, tight IAM integration, and reduced control-plane complexity matter more than Kubernetes portability or ecosystem requirements.

4. How do capacity providers improve ECS design?

They decouple service placement from raw launch-type choices and allow weighted/base strategies plus managed scaling for EC2-backed capacity or Fargate/Fargate Spot selection.

Chapter Summary

- ECS separates workload specification, running tasks, desired-state services, clusters, and capacity.

- Execution roles and task roles must remain separate.
- Private tasks behind a load balancer, digest-pinned images, health-aware rollback, and service-pressure scaling form the production baseline.
- Service events, stopped reasons, logs, target health, and capacity metrics are the primary diagnostic evidence.

Official AWS References

Amazon ECS Developer Guide: <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html>

ECS task definitions: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task_definitions.html

ECS capacity providers: <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/capacity-launch-type-comparison.html>

VOLUME 6D | CHAPTER 33

Chapter 33 - AWS Fargate Serverless Containers

Running isolated tasks without operating worker instances

Learning Objectives

- Explain the Fargate responsibility model for ECS and EKS.
- Choose task sizing, networking, ephemeral storage, platform version, and capacity strategy.
- Design secure private-subnet connectivity and secrets access.
- Use Fargate and Fargate Spot appropriately.
- Diagnose startup, networking, platform, and cost problems.

33.1 What Fargate Removes - and What It Does Not

AWS Fargate provides on-demand, right-sized compute capacity for containers. Teams do not provision or patch worker instances, manage cluster packing, or scale EC2 Auto Scaling groups. The application team still owns image security, task or pod configuration, IAM, network paths, runtime health, dependency behavior, observability, scaling policies, and cost efficiency.

You no longer manage	You still manage
Worker AMIs, host patching, instance replacement	Container base images, dependencies, application vulnerabilities
EC2 capacity groups and bin packing	CPU/memory sizing, task count, scaling signals, workload architecture
Host SSH and daemon maintenance	Runtime logs, metrics, traces, health checks, incident diagnosis
Host-level tenant isolation decisions	Task/pod IAM, security groups, secrets, network egress, data access

33.2 Sizing and Runtime Characteristics

Fargate tasks use supported CPU and memory combinations. Oversizing increases steady cost; undersizing causes CPU throttling, memory termination, latency, and retry storms. Establish load-test profiles for startup, steady state, bursts, garbage collection, sidecars, and failure recovery. Ephemeral storage is temporary task storage and must not be treated as durable state.

Design principle Externalize durable state to managed data services. A Fargate task should be disposable: replacement must not require local recovery or manual host access.

33.3 Private Networking

A private Fargate task needs network reachability to every bootstrap and runtime dependency. Image layers, logs, secrets, configuration, telemetry, and external APIs may require NAT or VPC endpoints. A task that cannot reach ECR, S3-backed image layers, CloudWatch Logs, Secrets Manager, or KMS can fail before application code starts.

Dependency	Private connectivity option	Failure evidence
ECR API and registry	Interface VPC endpoints or NAT	ResourceInitializationError or image-pull timeout
Image layer storage	S3 gateway endpoint or NAT	Layer download timeout
CloudWatch Logs	Logs interface endpoint or NAT	Log driver initialization failure or absent streams
Secrets Manager / Parameter Store	Interface endpoint or NAT plus KMS access	Secret retrieval access denied or timeout
Application dependencies	Private endpoint, peering/TGW, or controlled egress	Application health-check failure or connection timeout

33.4 Capacity Strategies

For ECS, FARGATE provides regular serverless capacity and FARGATE_SPOT can reduce cost for interruption-tolerant tasks. A capacity provider strategy can set a base amount on regular capacity and distribute additional tasks by weight. Spot is appropriate only when tasks drain gracefully, tolerate interruption, and recover from durable queues or desired state.

33.5 When Fargate Is a Strong Fit

- Stateless APIs, workers, scheduled tasks, and event consumers with predictable task sizing.
- Teams that want container packaging but do not need host control, privileged containers, custom AMIs, or specialized accelerators.
- Spiky services where per-task scaling is more valuable than maximizing instance utilization.
- Security-sensitive environments that prefer task isolation and reduced host-management surface.

33.6 When EC2 Capacity May Be Better

- Very high steady utilization where optimized reserved or Spot EC2 fleets provide lower unit economics.
- Workloads requiring host-level agents, privileged mode, special kernel capabilities, custom AMIs, accelerators, or unusual storage/networking.
- Large numbers of tiny services where task-level overhead and billing granularity are inefficient.
- Cases where deterministic capacity reservations or bespoke placement are required.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
ResourceInitializationError	Bootstrap dependency unreachable, secret permission failure, or log configuration error.	Read stopped reason; test endpoint/NAT routes, SGs, DNS, execution-role and KMS permissions.	Restore network path and least-privilege permissions; retry with known-good task revision.
Task killed for memory	Memory hard limit too low or application leak/peak.	Inspect exit code, memory metrics, heap/GC logs, and load profile.	Fix leak, tune runtime, increase memory, or split workload.
Cost higher than EC2 design	Tasks are oversized, always-on, or poorly scaled.	Compare requested versus used CPU/memory and steady duty cycle.	Right-size, scale to demand, use Spot for tolerant work, or move steady fleet to EC2.

Symptom	Likely cause	Investigation	Corrective action
No public outbound connectivity	Private subnet lacks NAT/endpoints or SG/NACL/DNS path.	Test name resolution, routes, endpoints, flow logs, and SG egress.	Add approved private endpoints or controlled NAT egress.

Hands-on Lab 33: Run a Queue Worker on Fargate

Objective Deploy a Fargate worker that consumes durable jobs, scales on queue backlog, and survives task replacement without message loss.

Prerequisites

- Private subnets, approved egress or required VPC endpoints
- An ECR worker image
- An SQS standard queue and dead-letter queue
- IAM permissions for ECS, SQS, logs, and scaling

Implementation

18. Create the queue and DLQ with encryption and a redrive policy.
19. Register a Fargate-compatible task definition with task role limited to the queue.
20. Create an ECS service with zero or low minimum tasks and Fargate capacity provider strategy.
21. Publish test jobs and verify the worker deletes only successful messages.
22. Create a scaling metric based on visible backlog per running task.
23. Terminate a task while it processes a job and verify redelivery/idempotency.
24. Send a poison message and confirm DLQ routing after the configured receive count.
25. Run a cost and right-sizing review using observed CPU and memory.

Validation Evidence

- Queue depth and age metrics
- Successful and duplicate-safe processing evidence
- Task replacement events
- DLQ message and remediation record
- Right-sizing recommendation

Cleanup

- Purge and delete queues after evidence is captured.
- Delete service, task definition dependencies, logs, roles, and networking resources created only for the lab.

Stretch challenge Use a mixed FARGATE/FARGATE_SPOT strategy and verify graceful termination handling and replay from the queue.

Interview Questions and Model Answers

1. Is Fargate the same as ECS?

No. ECS is an orchestrator. Fargate is a serverless container compute option usable with ECS and EKS. ECS can also schedule onto EC2 or other supported capacity.

2. Why can a private Fargate task fail before the application starts?

The platform must pull images, retrieve secrets, and initialize logs. Missing routes, endpoints, DNS, security-group rules, KMS permissions, or execution-role permissions can block bootstrap.

3. What makes a workload safe for Fargate Spot?

Durable state, idempotent processing, graceful termination handling, retry/backoff, capacity diversity, and an SLO that tolerates interruption.

4. How should Fargate tasks store data?

Use ephemeral storage only for transient scratch space and externalize durable state to services such as S3, databases, queues, or shared filesystems where appropriate.

Chapter Summary

- Fargate removes worker-instance operations but not application, identity, network, observability, or cost responsibilities.
- Task sizing and private dependency connectivity are the dominant design concerns.
- Disposable stateless tasks and durable external state enable safe replacement and Spot usage.
- Right-size from measured load and compare economics against optimized EC2 capacity.

Official AWS References

Fargate for Amazon ECS: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html

Fargate for Amazon EKS: <https://docs.aws.amazon.com/eks/latest/userguide/fargate.html>

VOLUME 6D | CHAPTER 34

Chapter 34 - Amazon EKS and Kubernetes Operations

Managed Kubernetes with explicit platform ownership

Learning Objectives

- Explain the EKS control plane, data plane, Kubernetes API, add-ons, and AWS integrations.
- Choose managed node groups, Fargate, Auto Mode, or self-managed capacity according to constraints.
- Design identity, network, ingress, storage, scaling, upgrade, and multi-tenant controls.
- Operate clusters using declarative delivery, policy, observability, and safe upgrades.
- Troubleshoot scheduling, networking, identity, image, and application failures.

34.1 Why EKS Requires a Platform Model

Amazon Elastic Kubernetes Service operates the Kubernetes control plane, but a production cluster remains a distributed platform with shared APIs, networking, admission policy, add-ons, node capacity, identities, storage, observability, upgrades, and tenant boundaries. EKS reduces control-plane operations; it does not eliminate Kubernetes engineering.

Layer	AWS responsibility	Platform team responsibility
Control plane	Managed Kubernetes API availability and control-plane infrastructure.	Version selection, access, audit logging, API usage, upgrade sequencing, policy.
Compute	Managed options automate portions of node lifecycle.	Capacity model, instance families, node groups, Fargate profiles, disruption budgets, scaling.
Networking	VPC integration components and managed service endpoints.	CIDR/IP capacity, security groups, network policies, ingress/egress, DNS, service exposure.
Workloads	Kubernetes schedules declared resources.	Images, manifests, probes, resources, security context, autoscaling, SLOs, runbooks.
Add-ons	AWS supplies supported add-ons and integrations.	Version compatibility, configuration, rollout, monitoring, ownership.

34.2 Compute Options

Option	Strength	Trade-off
Managed node groups	AWS automates provisioning and lifecycle of EC2 nodes.	Team still chooses AMI family, instance types, scaling, upgrades, and workload placement.
EKS Auto Mode	Greater automation of compute and related infrastructure.	Requires compatibility review, governance, and clear understanding of managed behavior.

Option	Strength	Trade-off
Fargate	No worker-node management for selected pods.	Pod constraints, cost profile, and daemon/host capability limitations.
Self-managed nodes	Maximum customization and specialized capacity.	Highest operational responsibility and upgrade/security burden.

34.3 Identity and Access

Separate human access to the Kubernetes API from workload access to AWS services. Human access should use centralized federation, EKS access controls, and Kubernetes RBAC. Workloads should use pod-scoped AWS identities such as EKS Pod Identity or IAM roles for service accounts where appropriate. Node roles must not become a shared credential source for every pod.

Least-privilege rule A namespace is not an IAM security boundary by itself. Combine Kubernetes RBAC, pod identity, network policy, admission policy, and separate accounts or clusters where blast-radius requirements demand it.

34.4 Networking and IP Planning

The VPC CNI commonly assigns VPC-routable addresses to pods. Large clusters can exhaust subnet address space before CPU capacity is exhausted. Plan pod density, secondary address allocation, IPv6 where applicable, private endpoint access, control-plane connectivity, load balancers, service-to-service policy, CoreDNS capacity, and egress.

34.5 Workload Reliability

Control	Purpose	Failure if omitted
Requests and limits	Guide scheduling and constrain resource use.	Noisy neighbors, poor bin packing, OOM kills, and unpredictable scaling.
Readiness probe	Controls traffic eligibility.	Traffic reaches pods before startup or during dependency failure.
Liveness/startup probe	Detects stuck process while protecting slow startup.	Restart loops or undetected deadlocks.
PodDisruptionBudget	Limits voluntary disruption.	Node upgrades can remove too many replicas at once.
Topology spread / anti-affinity	Distributes replicas across failure domains.	Replicas concentrate on one node or AZ.
HPA/Karpenter/cluster scaling	Adds pods and capacity for demand.	Pending pods, latency, or overprovisioning.

34.6 Upgrade Strategy

Kubernetes upgrades are a compatibility program, not a console button. Inventory deprecated APIs, add-on versions, admission webhooks, ingress controllers, CSI drivers, operators, observability agents, and

client tooling. Upgrade non-production first, run conformance and business-path tests, then control node disruption with surge capacity and disruption budgets. Maintain rollback or replacement options for addons and applications.

```
kubectl get nodes -o wide
kubectl get pods -A
kubectl get events -A --sort-by=.lastTimestamp
kubectl describe pod <pod> -n <namespace>
kubectl auth can-i get secrets --as=<user> -n <namespace>
kubectl top nodes
kubectl top pods -A
```

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Pod Pending	Insufficient capacity, taints, affinity, PVC binding, IP shortage, or quota.	Describe pod; inspect scheduler events, nodes, taints, requests, PVC, and subnet IPs.	Add compatible capacity/IPs, correct placement rules, or resize requests.
ImagePullBackOff	Bad image reference, ECR permission, network path, or architecture.	Describe pod, check events, digest, node/pod identity, endpoint/NAT.	Fix reference, permissions, connectivity, or build platform.
Service has no endpoints	Selector mismatch or pods not Ready.	Compare Service selector and pod labels; inspect readiness.	Correct labels/selectors or readiness failure.
AWS API AccessDenied from pod	Pod identity not configured, trust mismatch, or IAM policy denied.	Inspect service account/pod identity association, role session, CloudTrail.	Correct association/trust/policy; never fall back to broad node permissions.
DNS intermittent	CoreDNS pressure, network policy, node issue, or upstream resolver.	Inspect CoreDNS replicas/logs/metrics, policies, node health, query latency.	Scale/tune CoreDNS, repair network path, and add capacity.

Hands-on Lab 34: Deploy and Operate an EKS Application

Objective Deploy a multi-replica application with pod-scoped AWS identity, ingress, autoscaling, disruption protection, and a controlled node rollout.

Prerequisites

- An approved EKS sandbox cluster or permission to create one
- kubectl, AWS CLI, and cluster creation tooling
- An ECR image and a private test data resource

Implementation

26. Create or select a cluster with private-aware networking and audit logs.
27. Create a namespace, resource quota, limit range, and RBAC roles.
28. Associate a pod-scoped IAM role that can read only the test data resource.

29. Deploy the application with digest-pinned image, security context, requests/limits, probes, and topology spread.
30. Expose it using an approved ingress/load-balancer pattern.
31. Configure HPA and generate controlled load.
32. Create a PodDisruptionBudget and perform a managed node-group rollout or controlled drain.
33. Verify availability, pod identity, autoscaling, and audit evidence.
34. Run policy checks against privileged containers, latest tags, and missing resources.

Validation Evidence

- Cluster/add-on versions and architecture export
- RBAC and pod-identity evidence
- Healthy replicas across failure domains
- Autoscaling and node-drain evidence
- Audit logs and policy results

Cleanup

- Delete application, ingress/load balancer, namespace, identities, and data resource.
- Delete the cluster and networking created solely for the lab to stop ongoing charges.

Stretch challenge Implement GitOps reconciliation and prove that a manual production manifest change is detected and restored to the approved state.

Interview Questions and Model Answers

1. What does EKS manage, and what remains your responsibility?

AWS manages the Kubernetes control-plane infrastructure. The customer owns access, workload manifests, data plane choices, add-ons, network and identity configuration, policies, upgrades, observability, and application reliability.

2. Why is a Kubernetes namespace insufficient for strong tenant isolation?

Namespaces organize and scope Kubernetes resources, but stronger isolation also requires RBAC, network policy, pod security/admission, IAM separation, quotas, secrets boundaries, and sometimes separate clusters or accounts.

3. How do you diagnose a Pending pod?

Start with scheduler events from `kubectl describe pod`, then check requests, taints/tolerations, affinity, topology, quotas, PVC binding, node capacity, autoscaler state, and subnet IP availability.

4. When is EKS preferable to ECS?

When Kubernetes APIs, ecosystem tooling, portability requirements, custom operators, complex scheduling, or organizational Kubernetes standards justify the added platform complexity.

Chapter Summary

- EKS is a managed control plane, not a fully managed application platform.
- Compute, identity, networking, add-ons, policy, upgrades, and workload reliability require explicit owners.
- Pod-scoped AWS identities, declarative delivery, resource controls, probes, disruption budgets, and topology distribution are baseline controls.
- Events and resource state usually reveal Kubernetes failures faster than application guessing.

Official AWS References

Amazon EKS User Guide: <https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html>

EKS compute resources: <https://docs.aws.amazon.com/eks/latest/userguide/eks-compute.html>

EKS managed node groups: <https://docs.aws.amazon.com/eks/latest/userguide/managed-node-groups.html>

VOLUME 6D | CHAPTER 35

Chapter 35 - AWS Lambda Production Engineering

Event-driven functions with concurrency, idempotency, and controlled failure

Learning Objectives

- Explain Lambda execution environments, invocation models, versions, aliases, layers, and event sources.
- Design concurrency, scaling, timeout, memory, storage, networking, and cold-start controls.
- Implement idempotency, retries, destinations, dead-letter handling, and partial batch failure.
- Secure execution roles, secrets, code, and network paths.
- Operate Lambda with logs, metrics, traces, alarms, and safe deployment strategies.

35.1 The Lambda Responsibility Model

AWS Lambda runs code in response to events without server provisioning. AWS manages the underlying compute fleet and execution environment lifecycle. The engineering team still owns function code, dependencies, runtime lifecycle, IAM, event contracts, concurrency, error semantics, network access, observability, cost, and downstream protection.

35.2 Invocation Models

Model	Examples	Error behavior
Synchronous	API Gateway, direct Invoke, application request.	Caller receives response/error and owns retry policy unless an intermediary handles it.
Asynchronous	EventBridge, SNS, S3 notifications, async Invoke.	Lambda queues the event, retries according to service behavior, and can route success/failure destinations.
Poll-based event source mapping	SQS, Kinesis, DynamoDB Streams, Kafka sources.	Lambda pollers batch records and manage scaling; consumer code must handle batch semantics and partial failures.

35.3 Concurrency as a Safety Control

Concurrency is the number of active function executions. Account concurrency is shared within a Region. Reserved concurrency guarantees and caps capacity for a function; provisioned concurrency pre-initializes environments for latency-sensitive paths. Concurrency must protect both the function and its dependencies. An unconstrained function can scale faster than a database connection limit, third-party API quota, or downstream write capacity.

```
Requests or events
-> invocation admission
-> function concurrency
-> downstream connection pool / API quota
-> business side effects

Safety controls:
reserved concurrency + event-source maximum concurrency
+ queue buffering + backoff + idempotency + downstream rate limits
```

35.4 Idempotency and Duplicate Delivery

Many event paths provide at-least-once delivery. A retry may occur after the business side effect succeeded but before acknowledgement was recorded. An idempotency key should represent the business operation, not only a random invocation. Store the result or completion state with an atomic conditional write and a retention policy appropriate to the replay window.

Payment example Use the order or payment-attempt identifier as the idempotency key. A duplicate event should return the existing result, not create a second charge.

35.5 Configuration and Deployment

- Use versions as immutable snapshots and aliases as controlled traffic pointers.
- Deploy with canary or linear traffic shifting and alarms on errors, latency, throttles, and business failures.
- Keep deployment packages small, remove unused dependencies, and use supported runtimes.
- Store secrets in managed services and retrieve/cache them safely; do not place secrets in environment variables without understanding visibility and rotation requirements.
- Set timeout below upstream timeout and above realistic execution time; configure SDK timeouts and retry budgets.

35.6 Networking

A function attached to a VPC can access private resources but must still reach public AWS or internet endpoints through approved endpoints or egress paths. VPC attachment is not an automatic security improvement; it adds routing, security-group, IP capacity, and dependency considerations. Keep functions outside a VPC when they need only public AWS service APIs that are securely accessible without private resources, subject to policy.

35.7 Queue Event Sources

For SQS, the queue visibility timeout must allow the function enough time to process and retry a batch. Use batch size and batching windows according to latency and efficiency. Configure partial batch response so successfully processed records are not repeatedly retried with failed records. Cap event-source concurrency to protect dependencies and monitor queue age, not only queue depth.

```
aws lambda get-function-configuration --function-name orders-handler
aws lambda get-function-concurrency --function-name orders-handler
aws lambda list-event-source-mappings --function-name orders-handler
aws logs tail /aws/lambda/orders-handler --since 15m --follow
```

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Function throttled	Account, reserved, provisioned, or event-source concurrency limit reached.	Inspect ConcurrentExecutions, Throttles, event source scaling, and downstream limits.	Increase approved capacity or reduce concurrency with buffering, optimization, and backpressure.
Timeouts with no clear error	Dependency latency, DNS/network issue, SDK retries, or insufficient timeout.	Use traces and structured timing logs; inspect VPC flow, dependency metrics, SDK config.	Fix dependency path, set bounded retries/timeouts, increase memory if CPU-bound, or redesign async.
SQS messages repeatedly return	Function failed whole batch, visibility timeout too short, or delete acknowledgement absent.	Inspect batch logs, ApproximateReceiveCount, visibility setting, partial batch config.	Implement partial failures/idempotency and align visibility timeout.
Sudden database connection exhaustion	Lambda concurrency exceeds database connection capacity.	Correlate Lambda concurrency, DB sessions, errors, and cold starts.	Cap concurrency, use a connection proxy/pool, reuse clients, and queue writes.

Hands-on Lab 35: Build an Idempotent Lambda Consumer

Objective Process SQS orders with idempotency, partial batch failure, controlled concurrency, structured logs, and DLQ recovery.

Prerequisites

- SQS source queue and DLQ
- A DynamoDB table or approved store for idempotency records
- IAM permission to create Lambda functions, roles, logs, and event source mappings

Implementation

35. Create an execution role limited to queue read/delete, idempotency table operations, logs, and required KMS actions.
36. Package a function that validates the event, claims an idempotency key atomically, executes the simulated side effect, and records the result.
37. Configure timeout, memory, environment configuration, log retention, and tracing.
38. Create the SQS event source mapping with batch size, partial batch response, and maximum concurrency.
39. Publish duplicate order events and verify one business result.
40. Inject one failing record in a mixed batch and verify successful records are not replayed.
41. Allow the poison record to reach the DLQ, repair it, and replay through an approved process.
42. Publish a version, route an alias, and perform a small canary update with alarm-based rollback criteria.

Validation Evidence

- Idempotency records and single side effect
- Partial batch response evidence
- Concurrency and queue-age metrics
- DLQ recovery record
- Version/alias and deployment evidence

Cleanup

- Disable and delete the event source mapping.
- Delete queues, function versions not required, table, logs, alarms, and roles.

Stretch challenge Add a function destination for asynchronous failures and correlate the original event, function version, and remediation workflow.

Interview Questions and Model Answers

1. What is the difference between reserved and provisioned concurrency?

Reserved concurrency both reserves and caps concurrent executions for a function. Provisioned concurrency keeps a configured number of execution environments initialized to reduce startup latency; it does not replace broader concurrency planning.

2. Why must Lambda consumers be idempotent?

Retries and at-least-once event delivery can invoke the same business operation more than once, including after a side effect succeeded. Idempotency prevents duplicate business outcomes.

3. How do you protect a relational database from Lambda scaling?

Use reserved or event-source maximum concurrency, queue buffering, connection reuse, a managed proxy or pool where appropriate, bounded retries, and capacity-aware SLOs.

4. What metrics matter for an SQS-triggered function?

Queue age and depth, function errors, throttles, duration, concurrency, batch failures, DLQ inflow, and business throughput. Age is often the clearest user-impact indicator.

Chapter Summary

- Lambda removes server operations but not event, concurrency, dependency, and failure engineering.
- Idempotency is mandatory for retryable event paths.
- Concurrency is both a performance mechanism and a downstream safety boundary.
- Versions, aliases, structured telemetry, partial batch failure, and durable recovery paths form the production baseline.

Official AWS References

AWS Lambda Developer Guide: <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>

Lambda concurrency: <https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html>

Using Lambda with SQS: <https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html>

VOLUME 6D | CHAPTER 36

Chapter 36 - Amazon API Gateway Engineering

Secure, observable contracts for synchronous application access

Learning Objectives

- Compare REST, HTTP, and WebSocket API products.
- Design resources/routes, stages, integrations, authorizers, throttling, validation, and custom domains.
- Protect backends using quotas, caching, WAF where supported, and bounded timeouts.
- Implement versioning, canary releases, observability, and contract testing.
- Troubleshoot authorization, integration, latency, mapping, and CORS failures.

36.1 API Gateway as a Boundary

Amazon API Gateway creates, publishes, monitors, and secures REST, HTTP, and WebSocket APIs. It sits at a synchronous trust boundary: untrusted clients send requests; the gateway authenticates, authorizes, validates, transforms, throttles, observes, and routes them to Lambda, AWS services, or HTTP backends. The API contract must be treated as a product with ownership, versioning, compatibility, and SLOs.

API product	Best fit	Important characteristics
HTTP API	Cost-sensitive RESTful APIs with straightforward routing and authorization needs.	Lower-cost, streamlined feature set; choose when advanced REST API features are unnecessary.
REST API	Feature-rich APIs needing capabilities such as detailed request validation, usage plans, API keys, private APIs, or other advanced controls.	More configuration and cost; supports mature API-management features.
WebSocket API	Two-way, stateful communication such as live updates or collaboration.	Routes connect, disconnect, and message actions; applications manage connection identifiers and fanout.

36.2 Authentication and Authorization

- IAM authorization is appropriate for AWS-aware callers that can sign requests.
- JWT or user-pool authorizers validate user identity and claims for supported API types.
- Lambda authorizers implement custom authorization but add code, latency, caching, and failure modes.
- Resource policies restrict source networks, VPC endpoints, accounts, or organizations where supported.
- API keys identify clients for usage and metering; they are not a substitute for authentication.

36.3 Backend Protection

A public API can create demand faster than a backend can absorb. Configure account and stage throttles, method or route controls, usage plans where applicable, Lambda concurrency, queue buffering for asynchronous operations, database capacity protection, and circuit-breaking behavior. A 429 response is often safer than allowing cascading failure.

Synchronous design rule Do not keep a client connection open for work that can be accepted and completed asynchronously. Return a request identifier, place durable work on a queue, and provide status or callback mechanisms.

36.4 Validation and Mapping

Validate path, query, header, and body requirements at the boundary when supported, but do not rely on gateway validation as the only application defense. Define canonical error envelopes, correlation IDs, content types, payload limits, and schema evolution rules. Mapping templates or parameter transformations can adapt external contracts to internal backends, but excessive transformation becomes hard-to-test integration logic.

36.5 Deployment and Domains

- Use separate stages or APIs for environments according to isolation policy; do not expose development backends through production domains.
- Map custom domains to controlled stages and automate certificate validation and DNS changes.
- Export or maintain OpenAPI definitions as version-controlled artifacts.
- Deploy canary traffic only with metrics and rollback rules.
- Document deprecation timelines and backward-compatible changes; avoid breaking clients silently.

36.6 Observability

Signal	Question answered
Access logs	Who called which route, with what status, latency, identity, and correlation ID?
Execution/integration logs	Did mapping, authorization, or backend invocation fail?
Latency and integration latency	Is delay in the gateway or backend?
4XX and 5XX metrics	Are clients invalid/unauthorized, or is the platform failing?
Backend telemetry	Did the request create the intended business outcome?

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
401/403 response	Missing/invalid identity, authorizer failure, resource policy, IAM signature, or backend denial.	Separate gateway authorization from integration response; inspect authorizer and access logs.	Correct token/signature/policy and least-privilege backend permissions.
502 Bad Gateway	Malformed Lambda proxy response, integration connection failure, or backend error.	Inspect execution logs, backend logs, response shape, VPC link/endpoint.	Return expected schema and repair integration/network path.

Symptom	Likely cause	Investigation	Corrective action
High latency	Cold starts, backend latency, authorizer delay, retries, or oversized payload.	Compare total latency and integration latency; trace dependency calls.	Optimize backend, cache authorizer/results where valid, reduce payload, or redesign async.
Browser CORS failure	Preflight or response headers missing/mismatched.	Inspect OPTIONS response and actual response headers/origin/method.	Configure explicit allowed origins/methods/headers and test through browser path.

Hands-on Lab 36: Publish a Secure Serverless API

Objective Create an authenticated HTTP or REST API backed by Lambda, implement validation, structured errors, throttling, logs, custom domain planning, and a canary release.

Prerequisites

- A Lambda function with versions and alias
- Permissions for API Gateway, IAM, logs, and optional DNS/certificate resources
- A test identity provider or IAM caller

Implementation

43. Define the API contract and error envelope in OpenAPI or equivalent version-controlled form.
44. Create routes for create, read, and health operations.
45. Configure Lambda proxy integration using an alias, not an unpublished function.
46. Configure authentication/authorization and least-privilege invoke permissions.
47. Enable access logs with request and correlation identifiers.
48. Configure throttling and test expected 429 behavior.
49. Test invalid payloads, unauthorized calls, backend errors, and successful requests.
50. Publish a new Lambda version and shift a small traffic percentage with rollback alarms.
51. Document custom-domain, DNS, certificate, and versioning design even if not provisioned.

Validation Evidence

- API definition and stage configuration
- Authorization test matrix
- Access logs and latency metrics
- Throttle test and recovery
- Canary deployment and rollback evidence

Cleanup

- Delete API stages, custom-domain resources, logs, authorizer resources, and test identities created for the lab.

Stretch challenge Add an asynchronous submission route that writes to SQS and returns 202 Accepted with a status resource.

Interview Questions and Model Answers

1. How do HTTP APIs and REST APIs differ?

Both expose RESTful interfaces, but HTTP APIs provide a streamlined lower-cost feature set. REST APIs support a broader API-management feature set. Selection should be requirement-driven, not based on naming.

2. Why is an API key not authentication?

An API key is generally a client identifier used for metering or usage plans. It does not prove a user or workload identity with the security properties of IAM, JWT, or another authentication mechanism.

3. How do you distinguish gateway latency from backend latency?

Compare API Gateway total latency with integration latency, then use backend traces and metrics. A large difference can indicate gateway/authorizer/mapping overhead; similar values point to the backend.

4. When should an API return 202 Accepted?

When the request is valid and accepted but completion is asynchronous. The response should provide an operation identifier and a way to observe final status or receive completion notification.

Chapter Summary

- API Gateway is a synchronous trust and traffic boundary.
- Select HTTP, REST, or WebSocket APIs from feature and contract requirements.
- Authorization, throttling, validation, logging, correlation, and backend protection must be designed together.
- Long-running work should usually move behind durable asynchronous interfaces.

Official AWS References

API Gateway Developer Guide: <https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html>

Choose REST or HTTP APIs: <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-vs-rest.html>

VOLUME 6D | CHAPTER 37

Chapter 37 - Amazon SQS and Amazon SNS Messaging

Durable decoupling, fanout, retries, and poison-message control

Learning Objectives

- Explain queues, topics, producers, consumers, visibility timeout, retention, deduplication, and fanout.
- Choose SQS standard or FIFO and SNS standard or FIFO patterns.
- Design DLQs, redrive, long polling, batching, filtering, encryption, and access policies.
- Implement idempotent consumers and backlog-based scaling.
- Troubleshoot duplicate, delayed, stuck, unauthorized, and poison-message behavior.

37.1 Queue Versus Topic

Service	Delivery model	Primary use
Amazon SQS	Consumers poll a durable queue; a message is processed by a consumer path.	Decouple producers and workers, buffer bursts, control concurrency, and preserve work during outages.
Amazon SNS	Publishers push to multiple subscriptions through a topic.	Fan out notifications/events to queues, functions, HTTP endpoints, and other supported subscribers.
SNS + SQS	Topic fans out into independent durable queues.	Each consumer domain gets its own backlog, retry, scaling, and failure isolation.

37.2 Standard and FIFO Queues

Standard queues provide very high scale and at-least-once delivery with best-effort ordering. FIFO queues preserve order within message groups and add deduplication semantics. FIFO does not remove the need for idempotent business logic: consumers, integrations, or side effects can still fail around acknowledgement boundaries.

Decision	Standard	FIFO
Ordering	Best effort	Strict within a message group
Duplicate delivery	Possible	Queue deduplication reduces duplicate enqueue behavior; consumers should still be idempotent
Parallelism	Broad independent scaling	Parallelism is bounded by active message groups and configured consumer concurrency
Use cases	Independent jobs, notifications, image processing, general buffering	Ordered commands, per-account sequencing, inventory operations requiring sequence

37.3 Message Lifecycle

```

Producer sends message
-> SQS stores redundantly
-> consumer receives batch
-> messages become invisible
-> consumer performs idempotent work
-> consumer deletes successful messages

If processing fails or visibility expires:
message becomes visible again
-> retry
-> after max receives, redrive to DLQ

```

Visibility timeout is not a processing deadline enforced by SQS. It is the period during which a received message is hidden from other consumers. Set it longer than normal processing plus retry uncertainty, extend it for legitimately long work, and stop extending it for stuck jobs so DLQ policy can take effect.

37.4 DLQ and Redrive Engineering

- A DLQ is a quarantine and diagnostic boundary, not a deletion bin.
- Choose maxReceiveCount from transient-retry behavior, business urgency, and processing time.
- Alarm on DLQ inflow and oldest message age.
- Preserve original message, failure context, schema version, and correlation identifiers.
- Redrive only after correcting the cause and confirming consumers are idempotent.
- Use controlled batch replay with rate limits so recovery does not recreate the incident.

37.5 SNS Fanout and Filtering

SNS topics replicate a publication to subscriptions. Subscription filter policies allow each consumer to receive only relevant messages based on attributes or message body configuration. Fanout to separate SQS queues gives each consumer independent durability, scaling, retention, and DLQ handling.

```

Order service publishes OrderCreated
-> SNS topic
    -> payment queue [filter: region/tenant supported]
    -> fulfillment queue
    -> analytics queue
    -> notification function

Each queue has its own consumer, visibility timeout, DLQ, and SLO.

```

37.6 Security

- Encrypt queues and topics according to data classification and grant KMS permissions to publishers, services, and consumers.
- Use identity policies and resource policies; an SNS-to-SQS subscription also needs a queue policy that permits the specific topic.
- Avoid sensitive information in resource names and message attributes that may appear in logs or metrics.
- Use VPC endpoints and organization/account conditions where private connectivity and governance require them.

- Validate cross-account principals and SourceArn/SourceAccount conditions to prevent confused-deputy access.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Messages processed more than once	Standard delivery, visibility too short, consumer timeout, or acknowledgement failure.	Check receive count, processing duration, visibility, consumer logs, and delete success.	Implement idempotency, align visibility, and handle partial batch success.
Queue age grows while consumers run	Consumer capacity insufficient, poison messages, downstream throttling, or poor batch settings.	Compare age/depth, concurrency, errors, duration, downstream metrics, DLQ.	Scale safely, isolate poison records, tune batching, and protect downstream.
SNS publishes but queue receives nothing	Subscription unconfirmed, filter mismatch, queue policy/KMS denial.	Inspect subscription attributes, delivery logs, filter scope, queue policy, KMS grants.	Confirm subscription and correct filter/resource/KMS policies.
FIFO throughput lower than expected	Too few message groups or one hot group serializes work.	Inspect group distribution and consumer concurrency.	Partition keys into safe independent groups or use standard queue if ordering is unnecessary.

Hands-on Lab 37: Implement Durable SNS-to-SQS Fanout

Objective Publish order events to an SNS topic and fan them out to filtered SQS queues with encryption, DLQs, idempotent consumers, and controlled replay.

Prerequisites

- Permissions for SNS, SQS, KMS, Lambda or ECS consumers, and CloudWatch
- Two consumer implementations or test scripts

Implementation

52. Create an encrypted SNS topic and two encrypted SQS queues, each with a DLQ.
53. Subscribe queues to the topic and configure queue policies scoped to the topic ARN.
54. Apply different filter policies to the subscriptions.
55. Publish matching and nonmatching events with correlation and schema identifiers.
56. Verify independent queue delivery and filtering.
57. Run idempotent consumers and test duplicate publication.
58. Inject a poison message, observe retries, and confirm DLQ redrive.
59. Correct the consumer and perform a rate-limited redrive.
60. Create alarms for oldest age, visible depth, and DLQ inflow.

Validation Evidence

- Topic/subscription and queue-policy exports
- Filter test matrix
- Duplicate-safe business result

- DLQ and replay evidence
- Alarm configuration and runbook

Cleanup

- Delete subscriptions, queues, DLQs, topic, test keys/aliases if dedicated, consumers, logs, and alarms.

Stretch challenge Build a FIFO topic-to-FIFO queue path and demonstrate ordering within two independent message groups.

Interview Questions and Model Answers

1. What is the purpose of SQS visibility timeout?

It temporarily hides a received message while a consumer processes it. If the consumer does not delete the message before visibility expires, the message can be delivered again.

2. Why combine SNS and SQS?

SNS provides fanout; SQS provides independent durable buffering and consumer backpressure. Together, each subscriber can process at its own rate and fail without blocking others.

3. How should you select maxReceiveCount?

From expected transient recovery, processing duration, retry cost, urgency, and poison-message risk. It should allow legitimate retries but move nonrecoverable messages to investigation before they consume excessive capacity.

4. What metric best reveals user delay in a queue system?

Age of the oldest message often maps directly to processing delay. Depth alone can be normal when throughput is high, while age shows whether work is meeting the SLO.

Chapter Summary

- SQS buffers and decouples work; SNS fans events to multiple subscribers.
- Standard queues require idempotency and tolerate reordering; FIFO provides ordered groups and deduplication features.
- Visibility timeout, DLQ, redrive, filtering, encryption, and resource policies are core production controls.
- Queue age, not only depth, should drive operations and scaling.

Official AWS References

Amazon SQS Developer Guide: <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html>

Amazon SNS Developer Guide: <https://docs.aws.amazon.com/sns/latest/dg/welcome.html>

SNS to SQS fanout: <https://docs.aws.amazon.com/sns/latest/dg/sns-sqs-as-subscriber.html>

VOLUME 6D | CHAPTER 38

Chapter 38 - Amazon EventBridge Integration Engineering

Routing domain events across services, accounts, and schedules

Learning Objectives

- Explain event buses, rules, patterns, targets, Pipes, Scheduler, archives, and replay.
- Design domain events with stable contracts, ownership, schema versions, and idempotency.
- Choose EventBridge, SNS, SQS, or Step Functions according to interaction semantics.
- Implement cross-account routing, filtering, enrichment, DLQs, and observability.
- Troubleshoot pattern, permission, target, retry, and schema failures.

38.1 Event Routing Versus Messaging

Amazon EventBridge routes events from AWS services, custom applications, and partner sources to targets using event patterns. An event bus is useful when producers should publish facts without knowing consumers and when routing depends on event metadata or detail. EventBridge Pipes provides point-to-point source-to-target integration with filtering, transformation, and optional enrichment. EventBridge Scheduler is the preferred dedicated service for large-scale one-time or recurring schedules.

Need	Best starting point
Durable work queue with consumer-controlled processing	SQS
Push one message to many known subscribers	SNS, often with SQS fanout
Route domain/AWS events by content across many targets/accounts	EventBridge event bus
Connect one supported source to one target with filter/enrichment	EventBridge Pipes
Coordinate a multi-step business process with state and branching	Step Functions
Invoke a target at a one-time or recurring schedule	EventBridge Scheduler

38.2 Event Contract Design

An event is a published fact. It should use a clear past-tense domain name, stable source, unique event identifier, occurrence time, subject or aggregate identifier, schema version, correlation/causation IDs, and data classification. Consumers should not infer mutable state from ambiguous fields. Breaking schema changes require a new version or compatibility strategy.

```
{
  "source": "com.example.orders",
  "detail-type": "OrderCreated.v1",
  "time": "2026-07-31T00:00:00Z",
  "detail": {
    "eventId": "...",
    "orderId": "...",
    "customerId": "...",
    "currency": "INR",
    "total": 1250,
    "correlationId": "..."
  }
}
```

38.3 Event Buses and Rules

Rules compare events against patterns and send matching events to targets. Patterns can match event source, metadata, and detail values. Use precise patterns to avoid accidental fanout and cost. Each target requires an IAM role or resource permission appropriate to that integration. Configure retry behavior and a dead-letter queue for failed target delivery where supported.

38.4 Cross-Account Architecture

A multi-account organization can centralize selected security or operational events while keeping application-domain buses decentralized. Resource policies control which accounts or organization entities can put events. Sender-side roles and service control policies preserve organization guardrails. Avoid a single unrestricted enterprise bus that becomes an ungoverned coupling point.

```
Application accounts
-> local domain event buses
-> approved cross-account rules
-> central security/operations bus
-> incident automation, audit lake, notifications

Domain consumers remain close to their owning application accounts.
```

38.5 Pipes and Scheduler

Pipes reduce integration code for a single supported source and target. A pipe can filter records, transform input, call an enrichment step, and send the result to a target. This is not a replacement for a many-subscriber event bus or stateful workflow. Scheduler supports cron, rate, one-time schedules, flexible time windows, retries, and target invocation at scale; it should be preferred over legacy scheduled rules for new scheduling use cases.

38.6 Archive and Replay

Archive and replay can help recover consumers or test new rules using historical events. Replay is not transactional rollback. Consumers must be idempotent, side effects must be understood, and replay scope/rate must be controlled. Preserve schema compatibility for the entire replay retention period or transform through a governed migration path.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Rule never matches	Event source/detail-type mismatch, nested field path wrong, or test event differs.	Capture actual event, use pattern test, compare exact case/types and nested structure.	Correct pattern and add contract tests with representative events.
Matched event not delivered	Target permission, role trust, target configuration, or throttling.	Inspect rule metrics, DLQ, CloudTrail, target logs/metrics and IAM.	Repair target policy/role, limits, and retry/DLQ configuration.
Duplicate business action after replay	Consumer is non-idempotent or replay scope overlaps processed events.	Trace event IDs, replay window, idempotency store, and side-effect records.	Add idempotency and controlled replay filters/rate.
Pipe backlog/target failure	Source polling, enrichment, transformation, target quota, or permissions.	Inspect pipe logs/metrics, source age, enrichment and target errors.	Correct filter/transform/permissions; throttle or queue target path.

Hands-on Lab 38: Build a Cross-Service Event Bus

Objective Publish domain events, route them using patterns, deliver failures to a DLQ, create a Pipe with enrichment, and test controlled archive replay.

Prerequisites

- Permissions for EventBridge, SQS, Lambda, IAM, logs, and optional cross-account bus policy
- A producer script and idempotent target function

Implementation

61. Create a custom event bus and resource tags.
62. Create an SQS target queue and DLQ.
63. Define a precise rule for OrderCreated.v1 events and configure target delivery.
64. Publish matching and nonmatching events and validate routing.
65. Create a Lambda target that records event IDs idempotently.
66. Introduce a permission failure and verify DLQ capture, then repair it.
67. Create a Pipe from a source queue to the bus with filtering and transformation or enrichment.
68. Create an archive, publish test events, and conduct a narrowly scoped replay.
69. Document schema ownership, compatibility, retention, and replay approval.

Validation Evidence

- Event samples and schema contract
- Rule pattern tests
- Target delivery and DLQ evidence
- Pipe filter/enrichment evidence
- Replay plan and duplicate-safe result

Cleanup

- Delete archive, rules, targets, pipe, queues, functions, bus, roles, logs, and test records.

Stretch challenge Create a cross-account destination bus policy limited to an organization or named account and prove unauthorized put-events calls are denied.

Interview Questions and Model Answers

1. How does EventBridge differ from SNS?

SNS is primarily topic-based push fanout to subscriptions. EventBridge provides content-based event routing through buses and rules, broader event-source integration, cross-account patterns, archive/replay, Pipes, and related event services.

2. When should you use EventBridge Pipes?

For a point-to-point integration from one supported source to one target where filtering, transformation, and optional enrichment can replace custom glue code.

3. Why must replay be governed?

Historical events may trigger real side effects again. Replay requires idempotent consumers, scope and rate controls, schema compatibility, approval, and clear understanding of downstream behavior.

4. What makes a good domain event?

It describes an immutable business fact, has clear ownership and version, includes unique identity and timing, avoids sensitive excess data, and supports correlation and compatibility.

Chapter Summary

- EventBridge routes events by content and supports buses, rules, Pipes, Scheduler, archives, and replay.
- Event contracts require ownership, schema compatibility, identifiers, and data classification.
- Use each integration service according to queueing, fanout, routing, scheduling, or orchestration semantics.
- DLQs, target permissions, contract tests, and idempotent replay are production controls.

Official AWS References

Amazon EventBridge User Guide: <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html>

EventBridge Pipes: <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-pipes.html>

Event patterns: <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-event-patterns.html>

VOLUME 6D | CHAPTER 39

Chapter 39 - AWS Step Functions Workflow Orchestration

Durable state machines for distributed business processes

Learning Objectives

- Explain Amazon States Language, state types, service integrations, and execution history.
- Choose Standard or Express workflows from duration, semantics, auditability, and cost.
- Implement retries, catches, timeouts, callbacks, parallel work, maps, and compensation.
- Control workflow data, payload size, IAM, observability, and versioning.
- Troubleshoot state, integration, data-shape, timeout, quota, and retry failures.

39.1 Orchestration Versus Choreography

Choreography lets services react to events without a central coordinator. Orchestration uses a workflow to make state, sequence, decisions, retries, and outcomes explicit. Choreography reduces central coupling but can make end-to-end progress difficult to observe. Orchestration improves visibility and control but can become a centralized business-logic bottleneck if every interaction is forced through one state machine.

Use orchestration when	Use choreography when
Sequence and branching are explicit business requirements.	Consumers can react independently to an event.
Human approval, callback, long wait, or compensation is needed.	There is no single authoritative process owner.
A complete execution history is operationally valuable.	Loose coupling and autonomous evolution are more important.
Failures require coordinated recovery across steps.	Each consumer owns its retry and recovery independently.

39.2 Standard and Express Workflows

Characteristic	Standard	Express
Best fit	Long-running, durable, auditable, non-idempotent or complex workflows	High-volume, short-duration event processing with idempotent steps
Duration	Can run for long periods, including waiting/callback use cases	Short-running workflows
Execution semantics	Exactly-once workflow execution unless Retry is configured	Asynchronous Express is at least once; synchronous behavior differs; design idempotent actions
History	Durable execution history and visual debugging	Relies heavily on configured logging for detailed history
Pricing model	State transitions	Executions, duration, and memory

39.3 State Types and Data Flow

Task states perform work through Lambda, AWS SDK service integrations, activities, or HTTP endpoints. Flow states include Choice, Parallel, Map, Pass, Wait, Succeed, and Fail. Each state receives JSON input and produces output. Input/output filtering is powerful, but large payloads and accidental sensitive-data propagation create cost, quota, and security problems. Store large objects in S3 and pass references.

```
{
  "StartAt": "ReserveInventory",
  "States": {
    "ReserveInventory": {
      "Type": "Task",
      "Resource": "arn:aws:states:::dynamodb:updateItem",
      "TimeoutSeconds": 10,
      "Retry": [{
        "ErrorEquals": ["DynamoDB.ProvisionedThroughputExceededException"],
        "IntervalSeconds": 2,
        "MaxAttempts": 4,
        "BackoffRate": 2.0
      }],
      "Catch": [{
        "ErrorEquals": ["States.ALL"],
        "Next": "ReservationFailed"
      }],
      "Next": "ChargePayment"
    },
    "ChargePayment": {"Type": "Task", "Resource": "...", "End": true},
    "ReservationFailed": {"Type": "Fail"}
  }
}
```

39.4 Retry and Error Design

Retries should target transient errors with bounded attempts, exponential backoff, and jitter where implemented by the called component. Do not retry validation failures or permanent business rejection. A Catch path should record context, choose compensation or manual review, and produce a terminal business status. Every task should define a timeout so a lost callback or stalled dependency cannot hold the workflow indefinitely.

Important Workflow-level exactly-once semantics do not guarantee that an external side effect occurs exactly once. A task can time out after a payment provider completed the charge. External operations still need idempotency keys and reconciliation.

39.5 Service Integration Patterns

- Request-response: invoke a service and continue after its API response.
- Run a job and wait (.sync): start a supported job and wait for completion.
- Callback with task token: pause until an external worker or human process returns the token.
- Map: process collections; Distributed Map supports large-scale parallel data processing in appropriate workflows.
- Nested workflows: decompose domains and combine Standard and Express patterns where justified.

39.6 Security and Observability

- Generate or define a workflow execution role with only the service actions and resource ARNs required by each state.
- Do not place secrets or regulated payloads in state input/output or logs. Pass references and retrieve under task-specific roles.
- Enable appropriate logging and tracing; Standard execution history and CloudWatch Logs answer different operational questions.
- Use deterministic execution names where they support duplicate prevention, but do not rely on names alone for business idempotency.
- Version state-machine definitions through Infrastructure as Code and deploy with change review and rollback paths.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Execution stuck in RUNNING	Missing timeout, waiting callback token, long Wait, or downstream job never completes.	Inspect current state, task heartbeat/timeout, callback registry, and downstream job.	Set timeout/heartbeat, reconcile token/job, and route to manual recovery.
States.Runtime / path error	Input path, result selector, or output path does not match actual JSON.	Inspect state input/output and definition paths using a safe test event.	Correct data transformation and add contract tests.
Retry storm	Permanent error included in Retry or attempts/backoff too aggressive.	Inspect error names, transition count, dependency metrics, and cost.	Retry only transient errors; add bounds, backoff, catch, and circuit-breaker path.
Duplicate side effect	Task retried after ambiguous timeout or Express at-least-once behavior.	Correlate execution/task attempts with external idempotency records.	Use idempotency key, reconciliation, and compensation.

Hands-on Lab 39: Orchestrate an Order Saga

Objective Build a Standard workflow that reserves inventory, charges payment, arranges fulfillment, sends notification, and compensates safely when a step fails.

Prerequisites

- Lambda functions or direct AWS service integrations for simulated steps
- DynamoDB tables or approved stores for workflow state and idempotency
- Permissions for Step Functions, IAM, logs, EventBridge, and integrations

Implementation

70. Define business states, terminal outcomes, compensation rules, and idempotency keys.
71. Create a Standard state machine using Task, Choice, Parallel, Wait, Succeed, and Fail states as appropriate.
72. Add bounded retries for transient errors and Catch paths for business/permanent failures.
73. Implement payment and inventory operations with idempotency.
74. Configure timeouts and a callback task for simulated manual approval.

75. Enable execution logging/tracing with sensitive-data controls.
76. Run success, inventory failure, payment timeout-after-success, approval timeout, and compensation scenarios.
77. Build a reconciliation query comparing workflow status and external side-effect records.
78. Version the definition in Infrastructure as Code and document rollback.

Validation Evidence

- State-machine definition and execution graphs
- Retry/catch and compensation evidence
- Idempotency and reconciliation records
- Timeout/manual recovery runbook
- Cost and workflow-type decision

Cleanup

- Delete state machine, versions/aliases where applicable, functions, tables, logs, roles, schedules, and test data.

Stretch challenge Create an Express child workflow for high-volume idempotent notification steps and invoke it from the Standard parent workflow.

Interview Questions and Model Answers

1. How do Standard and Express workflows differ?

Standard workflows suit durable, auditable, long-running processes and have exactly-once workflow execution semantics unless retries are configured. Express suits high-volume short workflows and requires careful idempotency and logging design.

2. What is a saga?

A saga coordinates a distributed business transaction as a sequence of local transactions with compensating actions when later steps fail, rather than relying on one global database transaction.

3. Why are timeouts required on Task states?

Without timeouts or heartbeats, a lost worker, callback, or dependency can leave an execution stuck and consume operational attention indefinitely.

4. Does Step Functions guarantee exactly-once payment?

No. Workflow semantics do not eliminate ambiguity in external side effects. Payment APIs need idempotency keys, status lookup, reconciliation, and compensation/refund processes.

Chapter Summary

- Step Functions makes distributed process state, decisions, retries, and recovery explicit.
- Standard and Express workflows serve different duration, audit, semantics, and cost requirements.
- Retries must be bounded and targeted; permanent failures need Catch and business recovery.
- Idempotency, reconciliation, compensation, timeouts, and sensitive-data control remain application responsibilities.

Official AWS References

AWS Step Functions Developer Guide: <https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html>

Choosing workflow type: <https://docs.aws.amazon.com/step-functions/latest/dg/choosing-workflow-type.html>

Step Functions best practices: <https://docs.aws.amazon.com/step-functions/latest/dg/sfn-best-practices.html>

VOLUME 6D | CHAPTER 40

Chapter 40 - Enterprise Event-Driven Application Architecture and Capstone

Integrating containers, functions, APIs, queues, events, and workflows into one operable system

Learning Objectives

- Select ECS, EKS, Fargate, and Lambda from workload and operating-model requirements.
- Combine API Gateway, SQS, SNS, EventBridge, and Step Functions without overlapping responsibilities.
- Design security, observability, resilience, deployment, data, and cost controls across service boundaries.
- Implement an enterprise order-processing capstone with failure injection and recovery evidence.
- Assess production readiness using architecture and operational quality gates.

40.1 The Reference System

The capstone is a multi-channel commerce order platform. Clients submit orders through a public API. The synchronous path validates identity and request shape, writes an order command, and returns an operation identifier. A durable workflow reserves inventory, processes payment, starts fulfillment, and records final status. Domain events fan out to analytics, notification, fraud, and support consumers. Container services handle steady APIs and specialized workers; Lambda handles event-driven glue and bursty transformations.

```

Clients
-> CloudFront/WAF as required
-> API Gateway
-> request Lambda or ECS API
-> SQS order-command queue
-> Step Functions Standard order saga
    -> inventory service
    -> payment adapter
    -> fulfillment service
    -> order status store
-> EventBridge domain bus
    -> SNS/SQS notification fanout
    -> analytics consumer
    -> fraud consumer
    -> operations events

Container images: ECR, digest-pinned, scanned and promoted
Runtime: ECS/Fargate or EKS according to platform decision
Telemetry: logs + metrics + traces + business events + audit trail
  
```

40.2 Workload Placement Decision

Workload	Preferred starting point	Reason
Public request validation and lightweight command acceptance	Lambda behind API Gateway	Bursty, stateless, short execution; fast independent scaling.
Steady order-query API with connection pooling	ECS/Fargate or ECS/EC2	Long-running service, predictable latency, connection reuse.
Kubernetes-standard fraud model service	EKS if ecosystem/portability requirement is real	Kubernetes API, operators, or organization platform standard.
Image/PDF transformation job	Fargate task or Lambda depending on duration/resources	Choose function for short event-driven work; container task for longer or custom runtime.
Order saga	Step Functions Standard	Durable state, retries, waits, compensation, execution history.
Independent domain consumers	EventBridge to SQS queues	Content routing plus durable per-consumer backpressure.

40.3 Security Architecture

- Use separate AWS accounts for production, non-production, security, logging, and shared services according to organization policy.
- Federate human access; use role sessions and least privilege. Workloads receive task, pod, or function roles scoped to resources.
- Keep compute in private subnets unless direct public addressing is justified. Expose through managed edges and load balancers/API Gateway.
- Encrypt data and messages, use TLS, centralize secrets, rotate credentials, and prohibit secrets in images and environment files.
- Sign or attest release digests, scan images, block mutable production tags, and record deployment provenance.
- Apply resource policies with SourceArn, SourceAccount, VPC endpoint, account, or organization conditions where appropriate.

40.4 Reliability Architecture

Failure	Containment and recovery
Traffic spike	API throttling, queue buffering, autoscaling, reserved concurrency, and backlog SLO.
Poison event	Validation, bounded retry, DLQ, quarantine, repair, and controlled redrive.
Payment ambiguity	Idempotency key, provider status query, reconciliation, and compensating refund/cancel path.
AZ or task failure	Multi-AZ service placement, load balancing, durable queue, replacement, and topology distribution.
Downstream outage	Circuit breaker, queue backlog, workflow retry/catch, timeout, and degraded customer status.

Failure	Containment and recovery
Bad deployment	Immutable version, canary/blue-green, health alarms, automatic rollback, and previous digest retention.
Region disaster	Defined RPO/RTO, replicated artifacts/data/events where justified, tested failover and reconciliation.

40.5 Observability Model

Technical telemetry must follow one business transaction across synchronous and asynchronous boundaries. Generate a correlation ID at entry and propagate it through API logs, queue attributes or event detail, workflow input, container logs, traces, and domain events. Business metrics such as order acceptance, completion time, payment success, compensation rate, and backlog age determine whether the service is delivering value.

Telemetry layer	Examples
Edge/API	request count, auth failures, 4XX/5XX, latency, throttle, route
Compute	task/pod/function errors, duration, saturation, restarts, concurrency
Messaging	oldest age, visible/in-flight messages, DLQ inflow, publish/delivery failure
Workflow	execution success/failure, state duration, retries, compensation, stuck executions
Business	order completion SLO, duplicate prevention, payment reconciliation, fulfillment delay
Audit/security	IAM changes, repository policy changes, unusual image pushes, resource policy denial

40.6 Delivery Pipeline

```
Commit
-> static analysis + unit tests
-> build container/function artifact
-> SBOM + scan + policy gate
-> integration/contract tests
-> push immutable digest / publish version
-> deploy to non-production
-> resilience and load tests
-> approval
-> canary or blue/green production release
-> automated health gates
-> progressive promotion or rollback
-> evidence archive and post-deployment verification
```

40.7 Cost Governance

- Measure cost per business transaction, not only monthly service totals.

- Right-size Fargate tasks, Lambda memory/duration, EKS nodes, log retention, API type, and workflow type.
- Use queues and schedules to scale idle workers down where SLOs permit.
- Control NAT and cross-AZ data processing by understanding traffic paths and using appropriate VPC endpoints.
- Set budgets, anomaly detection, ownership tags, expiry automation, and architecture cost reviews before production.
- Do not optimize away redundancy, telemetry, security, or recovery without documenting the SLO and risk trade-off.

40.8 Capstone Acceptance Criteria

Quality gate	Acceptance evidence
Security	No static credentials; scoped workload roles; encrypted data; private compute; image scan/provenance; authorization tests.
Reliability	Multi-AZ services; idempotent consumers; DLQs; bounded retry; compensation; rollback; tested task/pod/function failure.
Performance	Load profile, scaling evidence, latency and backlog SLOs, dependency protection, right-sizing.
Operations	Dashboards, alarms, trace correlation, runbooks, ownership, on-call escalation, audit trail.
Delivery	Infrastructure as Code, immutable artifacts, contract tests, progressive deployment, automated rollback gates.
Cost	Budget, tags, expiry, unit-cost estimate, idle-resource review, data-transfer analysis.

Hands-on Lab 40: Enterprise Order Platform Capstone

Objective Design, deploy, validate, break, recover, and document an enterprise order platform that uses the services in Chapters 31-39 with clear service boundaries.

Prerequisites

- A sandbox multi-account environment or a single training account with strong naming/isolation
- Infrastructure as Code tooling
- Source repositories for API, worker, and function components
- A documented budget and cleanup owner

Implementation

79. Create an architecture decision record for ECS/EKS/Fargate/Lambda placement and messaging choices.
80. Build ECR repositories with immutable promotion, scans, and lifecycle controls.
81. Deploy a synchronous order-entry API that validates identity and returns a correlation/operation ID.
82. Persist the command to SQS and use a Standard Step Functions saga for inventory, payment, fulfillment, and compensation.
83. Publish domain outcomes to EventBridge; route to SNS/SQS fanout consumers with filters and DLQs.

84. Deploy at least one container service and one Lambda consumer with scoped identities and private dependency access.
85. Implement end-to-end correlation, structured logs, metrics, traces, dashboards, and alarms.
86. Perform load testing and prove API throttling, queue buffering, safe consumer scaling, and downstream protection.
87. Inject task failure, function error, poison message, target permission denial, payment ambiguity, and bad deployment.
88. Execute runbooks for rollback, DLQ repair/redrive, workflow reconciliation, and service recovery.
89. Produce a security review, cost estimate, SLO report, evidence pack, and final cleanup plan.

Validation Evidence

- Architecture diagram and decision records
- IaC repository and release provenance
- Success and failure transaction traces
- Autoscaling/throttle/backlog test results
- Rollback, compensation, DLQ, and reconciliation evidence
- Security and cost review
- Runbooks and production-readiness sign-off

Cleanup

- Export all evidence and state snapshots required for assessment.
- Destroy load balancers, clusters, NAT gateways, tasks, functions, queues, topics, buses, state machines, logs, test keys, data stores, and temporary IAM resources.
- Verify Cost Explorer/billing views after service deletion and close residual findings.

Stretch challenge Extend the platform to a second Region with an explicit RPO/RTO, artifact replication, data strategy, failover exercise, and post-failover reconciliation.

40.9 Cross-Service Troubleshooting Matrix

Observed symptom	Trace the path	Likely control
Client timeout but order later completes	API request -> Lambda/ECS -> queue -> workflow	Use asynchronous 202 contract, shorter synchronous timeout, status endpoint.
Orders accepted but completion age rises	Queue age -> consumer concurrency -> workflow state -> dependency	Backlog scaling, dependency protection, poison isolation, SLO alarm.
Duplicate payment	Event ID -> workflow retries -> provider idempotency record	Idempotency key, reconciliation, compensation/refund.
New deployment receives no traffic	API alias/ALB target/ingress -> health -> security group	Traffic pointer, target health, listener rule, readiness, network rule.
Events missing from one consumer	Bus rule/SNS filter -> queue policy/KMS -> DLQ	Pattern/filter contract, resource policy, encryption permissions.
Unexpected cost spike	Traffic -> logs -> NAT/cross-AZ -> task/function duration -> workflow transitions	Retention, endpoints/traffic locality, right-sizing, workflow and API choice.

40.10 Part D Interview Board

1. How would you choose between Lambda and Fargate?

Use Lambda for event-driven, short, rapidly scaling functions with supported runtime constraints. Use Fargate when container control, longer execution, custom runtime dependencies, steady processes, or explicit task sizing are stronger requirements.

2. How would you choose between ECS and EKS?

ECS offers simpler AWS-native orchestration. EKS provides Kubernetes APIs and ecosystem flexibility but requires a platform operating model. The decision should follow organizational capability and workload requirements.

3. How do you prevent asynchronous systems from hiding failure?

Define backlog-age SLOs, DLQs, execution histories, correlation IDs, business status, reconciliation, alarms, ownership, and tested recovery paths.

4. What is the difference between retry and recovery?

Retry repeats an operation for a transient failure. Recovery restores correct business state after retries are exhausted or outcomes are ambiguous, using DLQ repair, reconciliation, compensation, or manual intervention.

5. Why is idempotency a cross-cutting requirement?

Queues, events, functions, workflows, networks, and external APIs can all retry. Idempotency ensures repeated delivery or ambiguous acknowledgements do not create duplicate business outcomes.

6. What does production readiness mean for serverless?

It means controlled identity, event contracts, concurrency, dependency protection, idempotency, retries/DLQs, observability, deployment rollback, cost limits, and runbooks - not merely absence of servers.

Chapter Summary

- Containers and serverless are complementary workload models.
- Integration services must have distinct responsibilities: queues buffer, topics fan out, buses route, and workflows orchestrate.
- End-to-end reliability depends on idempotency, bounded retry, durable recovery, correlation, and business-state reconciliation.
- Security, delivery, observability, cost, and cleanup are architecture components, not post-deployment additions.

Official AWS References

AWS Serverless Applications Lens: <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/welcome.html>

AWS Containers documentation: <https://aws.amazon.com/containers/>

AWS Architecture Center: <https://aws.amazon.com/architecture/>

Part D Command Reference

Area	Representative command
Identity	<code>aws sts get-caller-identity</code>
ECR	<code>aws ecr describe-images --repository-name <repo></code>
ECS service	<code>aws ecs describe-services --cluster <cluster> --services <service></code>
ECS tasks	<code>aws ecs describe-tasks --cluster <cluster> --tasks <task-arn></code>
EKS	<code>aws eks describe-cluster --name <cluster></code>
Kubernetes	<code>kubectrl get events -A --sort-by=.lastTimestamp</code>
Lambda	<code>aws lambda get-function-configuration --function-name <name></code>
API Gateway v2	<code>aws apigatewayv2 get-apis</code>
SQS	<code>aws sqs get-queue-attributes --queue-url <url> --attribute-names All</code>
SNS	<code>aws sns list-subscriptions-by-topic --topic-arn <arn></code>
EventBridge	<code>aws events list-rules --event-bus-name <bus></code>
Step Functions	<code>aws stepfunctions describe-execution --execution-arn <arn></code>
Logs	<code>aws logs tail <log-group> --since 15m --follow</code>
Cleanup verification	<code>aws resourcegroupstaggingapi get-resources --tag-filters Key=Expiry,Values=<date></code>

Production Readiness Checklist

- ☐ Artifacts are immutable, digest-pinned, scanned, traceable, and retained for rollback.
- ☐ Runtime roles are distinct from execution/bootstrap roles and are least privilege.
- ☐ Compute runs in approved network zones with explicit ingress and egress paths.
- ☐ Every service has ownership, SLOs, scaling limits, dependency budgets, and runbooks.
- ☐ All asynchronous consumers are idempotent and have bounded retry, DLQ, alarm, and redrive procedure.
- ☐ APIs have authentication, authorization, validation, throttling, logs, versioning, and contract tests.
- ☐ Events have owner, source, type, schema version, unique ID, correlation, and data classification.
- ☐ Workflows have timeouts, transient-only retries, catches, compensation, reconciliation, and sensitive-data controls.
- ☐ Deployments use versions/digests, progressive traffic, health gates, and tested rollback.
- ☐ Logs, metrics, traces, audit events, and business outcomes correlate through one transaction identifier.
- ☐ Costs are tagged, budgeted, measured per transaction, and reviewed for idle, data transfer, logs, and over-sizing.
- ☐ Failure injection and recovery evidence covers compute loss, poison messages, permission denial, dependency outage, and bad release.

Cross-Service Failure Triage

Layer	First evidence	Key question
Edge/API	Access log, 4XX/5XX, latency, authorizer result	Did the request enter the system and was it authorized?
Compute	Task/pod/function status, logs, concurrency, health	Did code run, and was it healthy and resourced?
Network	Routes, SGs, DNS, endpoints, flow logs	Could the component reach its dependency?
Messaging	Queue age/depth, DLQ, publish/delivery metrics	Is work buffered, stuck, filtered, or repeatedly failing?
Event routing	Pattern test, matched events, target failure/DLQ	Did the event match and could the target be invoked?
Workflow	Current state, input/output, retry/catch history	Where is the process and what recovery path was selected?
Business state	Order/payment/fulfillment records and reconciliation	Did the intended outcome occur exactly once?

Glossary

Term	Definition
Alias	A named pointer used to route traffic to a Lambda version or other versioned deployment target.
ASL	Amazon States Language, the JSON-based language used to define Step Functions workflows.
Capacity provider	An ECS mechanism that associates services with compute capacity and capacity strategy.
Choreography	Distributed coordination in which services react to events without one central workflow.
Cold start	Initialization work required before a new function execution environment handles an invocation.
Container digest	A cryptographic content identifier for an immutable image.
DLQ	Dead-letter queue used to quarantine messages or target deliveries that exceeded retry policy.
Event bus	An EventBridge resource that receives events and evaluates rules.
Event source mapping	A Lambda resource that polls queue or stream sources and invokes a function with batches.
Idempotency	Property that repeated processing of the same business operation produces one intended result.
Image layer	Content-addressed filesystem change that forms part of a container image.

Term	Definition
Message group	FIFO grouping key that preserves order within an independent sequence.
Orchestration	Explicit coordination of steps, decisions, state, retries, and recovery by a workflow.
Pipe	EventBridge point-to-point integration with filtering, transformation, and optional enrichment.
Provisioned concurrency	Pre-initialized Lambda execution environments for predictable startup latency.
Readiness probe	Kubernetes health check that controls whether a pod receives traffic.
Reserved concurrency	Lambda setting that reserves and caps concurrent executions for a function.
Saga	Distributed transaction pattern using local transactions and compensating actions.
Task definition	Versioned ECS workload specification for containers and runtime settings.
Visibility timeout	SQS period during which a received message is hidden from other consumers.

Official Documentation Index

Amazon ECR: <https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html>

Amazon ECS: <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html>

AWS Fargate: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html

Amazon EKS: <https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html>

AWS Lambda: <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>

Amazon API Gateway: <https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html>

Amazon SQS: <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html>

Amazon SNS: <https://docs.aws.amazon.com/sns/latest/dg/welcome.html>

Amazon EventBridge: <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html>

AWS Step Functions: <https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html>

AWS Well-Architected Framework: <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>

Serverless Applications Lens: <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/welcome.html>

Currency notice AWS services, quotas, runtime versions, Kubernetes versions, pricing, regional support, and recommended features change. Validate production decisions against the current official documentation and your organization standards.